

GCP Setup & Permissions Checklist

A self-serve punch list to close the remaining gaps in the cloud pipeline — the n8n identity-token fix, the hub-next deployment that doesn't exist yet, and everything already done that just needs confirming. Full audit for Path B of the migration runbook.

PROJECT	PROJECT NUMBER	PRIMARY REGION
molten-crowbar-498920-q8	137750788450	us-east4
PREPARED	SCOPE	RUNTIME SERVICE ACCOUNT
July 6, 2026	Full Path B — cloud migration runbook	137750788450-compute@developer.gserviceaccount.com

No new project. You already have the keys.

Two earlier docs in this repo (`ACCOUNT_REQUESTS.md` , `TECHNICAL_SPEC.md`) propose creating a fresh project named `id8-pipeline` and having an account owner grant you Owner on it. That never happened — everything already runs in `molten-crowbar-498920-q8` , and you've already been granting IAM bindings to service accounts yourself via Cloud Shell (Secret Manager access, Cloud SQL Client, etc. in `deploy.sh`). That means you already hold IAM-admin-level rights on this project.

ONE THING WORTH CONFIRMING, ONCE

Console → IAM & Admin → Settings — check the "Organization" field. If `molten-crowbar-498920-q8` sits inside the `id8investments.com` Workspace org, the OAuth consent screen can be set to **Internal** (Workspace-only sign-in) as every doc assumes. If it shows no organization, this is a standalone project outside the firm's Workspace — OAuth consent screen must be **External** instead, and Google may require app verification once you request sensitive scopes. Worth knowing before you hit it mid-deploy.

SECTION 1

APIs to enable

`deploy.sh` already enables the first five every run (it skips re-enabling if already on). The last three are net-new — nothing in the repo enables them yet.

API	USED BY	STATUS
<code>run.googleapis.com</code>	Cloud Run — id8 (Flask), n8n, cc-attio-sync	ENABLED
<code>sqladmin.googleapis.com</code>	n8n's Cloud SQL Postgres instance	ENABLED
<code>artifactregistry.googleapis.com</code>	Docker image storage for n8n build	ENABLED
<code>secretmanager.googleapis.com</code>	All API keys, DB password, encryption key	ENABLED
<code>cloudbuild.googleapis.com</code>	n8n image build, source deploys, hub build	ENABLED
<code>iamcredentials.googleapis.com</code>	Apps Script → Cloud Run identity-token minting (Section 2's 403 fix)	VERIFY / ENABLE
<code>firestore.googleapis.com</code>	hub-next's company-screen storage (Section 3)	ENABLE — NET NEW
<code>cloudscheduler.googleapis.com</code>	Weekly hub rebuild (optional, Section 6)	OPTIONAL

```
gcloud services enable iamcredentials.googleapis.com firestore.googleapis.com \
  --project=molten-crowbar-498920-q8
```

Finish the n8n identity-token fix

Commit `585fb5e` shipped the Apps Script code that mints a Cloud Run identity token by impersonating a dedicated service account — but the IAM side of that has to be done by hand in Cloud Console/gcloud; no script does it automatically. Until these bindings exist, the code will keep 403'ing exactly as before.

① Create the invoker service account (if it doesn't exist yet)

```
gcloud iam service-accounts create n8n-invoker \
  --display-name="Apps Script n8n invoker" \
  --project=molten-crowbar-498920-q8
```

② Let that service account call the n8n Cloud Run service

```
gcloud run services add-iam-policy-binding n8n \
  --region=us-east4 \
  --member="serviceAccount:n8n-invoker@molten-crowbar-498920-q8.iam.gserviceaccount.com" \
  --role="roles/run.invoker" \
  --project=molten-crowbar-498920-q8
```

③ Let your Apps Script identity impersonate that service account

Scoped to just this one service account, not the whole project. Use whichever Google account is logged into `script.google.com` when you run `installTriggers()` — the account that owns the Drive folders it watches, per `WEBHOOK_SETUP.md`.

```
gcloud iam service-accounts add-iam-policy-binding \
  n8n-invoker@molten-crowbar-498920-q8.iam.gserviceaccount.com \
  --member="user:YOUR_APPS_SCRIPT_ACCOUNT@id8investments.com" \
  --role="roles/iam.serviceAccountTokenCreator" \
  --project=molten-crowbar-498920-q8
```

④ Enable `iamcredentials.googleapis.com`

See Section 1 — this is the API the impersonation call (`generateIdToken`) actually hits.

⑤ Re-consent the Apps Script's new OAuth scope

`appscript.json` added the `cloud-platform` scope. In `script.google.com`, replace the manifest file (View → Show manifest file) with the repo's version, then run any trigger once by hand — Google will re-prompt for permission. Decline that prompt and you get silent 403s forever.

⑥ Test

Drop a file into a watched Drive folder, check the n8n Executions tab for the run within a few minutes (first hit after idle costs a 20–40s cold start).

Deploy hub-next

hub-next (added 2026-06-30) replaced the old Docusaurus/Firebase hub with a dynamic Next.js app — but no Dockerfile-to- Cloud-Run pipeline is wired up anywhere in the repo, it has no Firestore database to read from, and it needs its own OAuth client. It is not reachable today. Separately, pipeline/app.py 's /publish-hub route still rebuilds and deploys the old Firebase-hosted hub — that's a dead code path relative to hub-next, not a permissions issue, worth a look when you're back in the code (see Section 4).

1 Create the Firestore database — one-time, and the mode can't be changed later

```
gcloud firestore databases create --database="(default)" \
  --location=us-east4 --type=firestore-native \
  --project=molten-crowbar-498920-q8
```

2 Give the runtime service account read/write access

```
gcloud projects add-iam-policy-binding molten-crowbar-498920-q8 \
  --member="serviceAccount:137750788450-compute@developer.gserviceaccount.com" \
  --role="roles/datastore.user"
```

3 Create an Artifact Registry repo for hub-next images

```
gcloud artifacts repositories create hub-next \
  --repository-format=docker --location=us-east4 \
  --project=molten-crowbar-498920-q8
```

4 First deploy (from the hub-next/ directory)

```
gcloud run deploy hub-next --source . --region=us-east4 \
  --allow-unauthenticated --project=molten-crowbar-498920-q8
```

WATCH FOR THIS SPECIFICALLY

NextAuth enforces the @id8investments.com login gate itself, so this service is designed to stay public (--allow-unauthenticated). But if the org policy that blocked n8n's public access (Section 2's root cause) applies at the org level rather than being scoped to that one service, this deploy will fail the same way. Worth testing early — if it does fail, you're back to the invoker-service-account pattern from Section 2, or fronting this with IAP instead of relying on NextAuth.

5 Create a dedicated OAuth 2.0 Client ID for hub-next sign-in

Separate client from n8n's — Console → APIs & Services → Credentials → Create Credentials → OAuth client ID → Web application.

FIELD	VALUE
Authorized redirect URI	https://<hub-next-url>/api/auth/callback/google
Env vars to set (from hub-next/.env.example)	AUTH_GOOGLE_ID , AUTH_GOOGLE_SECRET , AUTH_SECRET (openssl rand -base64 32), ALLOWED_HD=id8investments.com , AUTH_URL , GCP_PROJECT_ID

Store the client ID/secret and `AUTH_SECRET` in Secret Manager and wire them in on redeploy with `--set-secrets`, same pattern as `deploy.sh` uses for `n8n`.

⑥ Confirm the OAuth consent screen is Internal

Needed for the `id8investments.com` -only sign-in to actually restrict at Google's layer (the app also double-checks the domain in `auth.js`, but Internal is the real gate). See the note on Cover / Section 0 about org membership.

SECTION 4 — OPTIONAL

Legacy Firebase hub — only if you're keeping it

Skip this whole section if `hub-next` fully replaces the old hub for you (the common case). It's listed here only because `pipeline/app.py` still auto-triggers it after every screening run — if you don't do this setup *and* don't repoint that code, `/publish-hub` will keep failing quietly on every screen.

- ☐ Enable `firebasehosting.googleapis.com`
- ☐ Link Firebase to the project via the Firebase console ("Add Firebase" to an existing GCP project) — this step has no `gcloud` equivalent
- ☐ Grant the Cloud Build default service account `roles/firebasehosting.admin` (`137750788450@cloudbuild.gserviceaccount.com`)
- ☐ Confirm the `GH_TOKEN` secret grants that same Cloud Build service account `roles/secretmanager.secretAccessor` — the inline build config in `app.py` pulls it via `availableSecrets`

SECTION 5 — CONFIRM, DON'T ASSUME

cc-attio-sync token rotation

`cc-attio-sync/deploy.sh` documents this exact binding in a comment but doesn't run it — worth confirming by hand, since the failure mode is silent (the sync keeps working until the current Constant Contact refresh token expires, then stops).

```
gcloud secrets add-iam-policy-binding CC_REFRESH_TOKEN \  
  --member="serviceAccount:137750788450-compute@developer.gserviceaccount.com" \  
  --role="roles/secretmanager.secretVersionAdder" \  
  --project=137750788450
```

Optional polish

WEEKLY HUB REBUILD

`CLOUD_INTELLIGENCE_PLAN.md` 's idea for keeping data-driven pages fresh. Needs `cloudscheduler.googleapis.com` plus a job hitting a rebuild trigger — hub-next has no such endpoint yet, so this is blocked on that code existing first.

CUSTOM DOMAIN

`intel.id8investments.com` → hub-next. Needs domain ownership verified in Search Console under whichever account manages DNS for `id8investments.com`, then `gcloud beta run domain-mappings create`. This is a DNS-access dependency, not a GCP IAM one.

Two things outside GCP that still gate Path B

ITEM	BLOCKS	OWNER
Attio custom fields — <code>fit_score</code> , <code>fit_gate</code> , <code>fit_rationale</code>	Phase 5 write-back (<code>DI_SLUG_*</code> in <code>deal_intelligence/config.py</code>)	Attio admin
OAuth authorized redirect URI for whichever n8n URL is currently live	Google Drive / Gmail re-authorization inside n8n	Console → Credentials